

TP n°4 ASP.NET Core MVC (.NET 8.0)

Code First Approach

Dans ce TP, vous allez travailler sur une solution ASP.NET Core MVC tout en vous basant sur l'approche de développement « Code First ».

Lancer Visual Studio Community 2022 et créer un nouveau projet.

Dans la première boîte qui apparaît, choisir "C#", "Toutes les plateformes" et "Web". Sélectionner "Application Web ASP.NET Core (modèle-vue-contrôleur)" et cliquer sur "Suivant".

Dans la deuxième boîte, attribuer le nom **Hotellerie-X** au projet et à la solution (X est votre nom), puis cliquer sur "Suivant".

Dans la dernière boîte, choisir ".NET 8.0" comme Framework, laisser les paramètres par défaut et cliquer sur "Créer".

A. Installation d'EntityFrameworkCore

Comme pour le TP2, commencer par l'installation d'Entity Framework Core et plus exactement des deux packages « **Microsoft.EntityFrameworkCore.SqlServer** » et « **Microsoft.EntityFrameworkCore.Tools** » à partir de Nuget: Menu **Outils – Gestionnaire de package Nuget – Gérer les packages Nuget pour la solution...**

NB: Installer la version 8.0.13.

I. Création du modèle

On souhaite développer une application de gestion hôtelière, pour cela :

1. Créez sous le dossier Models, un dossier "**HotellerieModel**".
2. Ajoutez dans ce nouveau dossier, la classe "**Hotel**" ayant les attributs suivants :
 - *Id* (int)
 - *Nom* (string)
 - champ obligatoire ayant un message d'erreur personnalisé;
 - longueur comprise entre 3 et 20 caractères avec un message d'erreur personnalisé.
 - *Etoiles* (int) : champ valant de 1 à 5 et ayant un message d'erreur personnalisé.
 - *Ville* (string) : champ obligatoire avec un message d'erreur personnalisé.
 - *SiteWeb* (string) : un champ obligatoire qui ne doit contenir que des URLs valides. Sur les vues, le nom de ce champ doit apparaître avec un espace « Site Web ».
3. Ajoutez dans le dossier **HotellerieModel**, la classe de contexte appropriée dite "**HotellerieDbContext**".

À rappeler que cette classe doit hériter de *DbContext* et représente une session vers la base de données. Elle contient un constructeur et un *DbSet* pour chaque classe de modèle (voir code ci-dessous). Ce *DbSet* est utilisé pour effectuer toutes les opérations CRUD.

Ici, on n'a besoin que d'un seul *DbSet* sur la classe « *Hotel* ».

```
public class HotellerieDbContext : DbContext
{
    public HotellerieDbContext(DbContextOptions options) : base(options)
    {
    }

    public virtual DbSet<Hotel> Hotels { get; set; }
}
```

NB : Pour générer le constructeur automatiquement, faites un clic droit dans le code de la classe, sélectionnez « Actions rapides et refactorisations... » puis « Générer le constructeur 'HotellerieDbContext(options)' ».

II. Configuration de la connexion à la BD

1. Ajoutez au fichier **appsettings.json** la chaîne de connexion « **HotellerieConnection** » vers la BD « **HotellerieDb** » sur le serveur « **(localdb)\mssqllocaldb** ».
2. Ajoutez le service correspondant à la classe de contexte et la chaîne de connexion à sa base dans le fichier **Program.cs** afin que l'application puisse se connecter à la base lors de son démarrage.

III. Activation de la Migration Code First

Afin de générer la BD à partir du modèle déjà créé, il faut lancer quelques commandes sur la console de Nuget (**Outils – Gestionnaire de package Nuget – Console du Gestionnaire du package**).

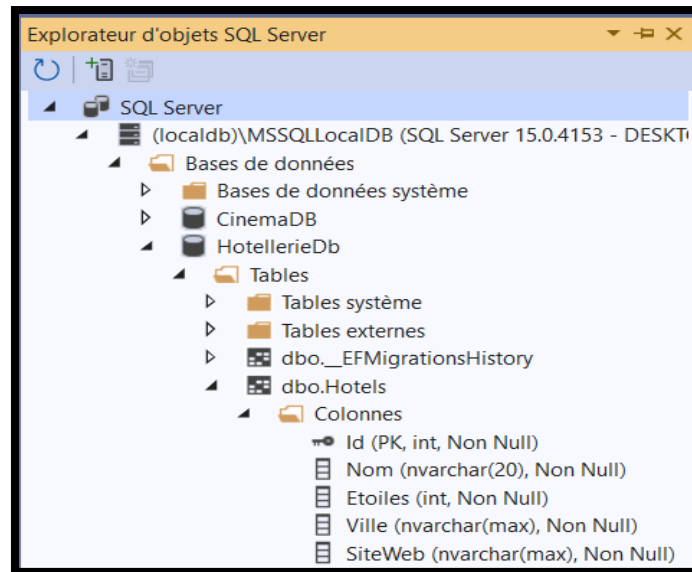
Sur cette console, vous allez exécuter :

1. La commande « **Add-Migration NomMigration** », par exemple « **Add-Migration InitialCreate** ».

Cette commande devrait activer la migration et créera un nouveau dossier "**Migrations**" contenant :

- un fichier *InitialCreate.cs* qui contient 2 méthodes **Up()** et **Down()** servant respectivement à mettre à jour la BD en Upgrade ou en Downgrade. Dans ce cas, la méthode **Up()** se charge de créer la table « *Hotels* » dans la BD, alors que la méthode **Down()** est responsable de la suppression de cette table.
 - un fichier *HotellerieDbContextModelSnapshot.cs* qui représente l'état actuel du modèle. Ce fichier est mis à jour avec chaque migration. Il permet au Framework de migrations de calculer les changements nécessaires pour synchroniser la base de données avec le modèle.
2. La commande « **Update-Database** » qui servira à créer la base de données avec les données de la migration ajoutée précédemment.

Une fois ces deux commandes exécutées avec succès, la BD sera visible sur l'explorateur d'objets SQL Server (voir figure suivante).



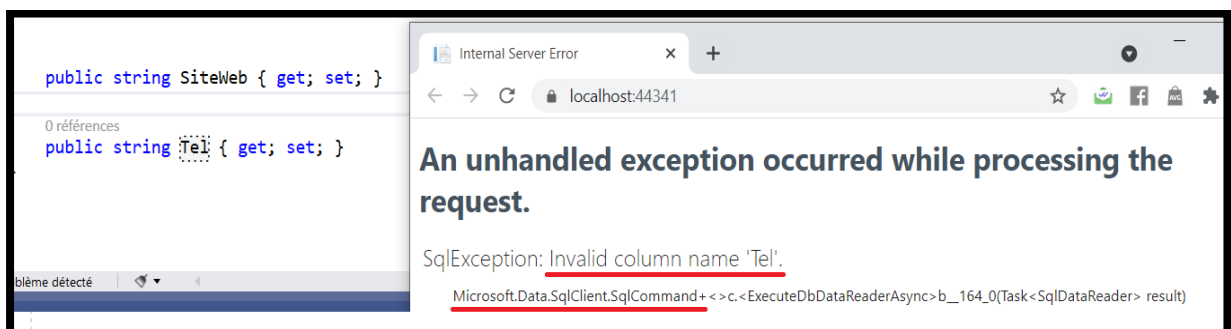
IV. Création du Contrôleur et des Vues

1. Ajoutez un contrôleur "**HotelsController**" avec les informations suivantes :
 - Contrôleur MVC avec vues, utilisant Entity Framework
 - **Classe du modèle** : *Hotel (HotellerieX.Models.HotellerieModel)*
 - **Classe de contexte** : *HotellerieDbContext (HotellerieX.Models.HotellerieModel)*
2. Définissez l'action Index de "**HotelsController**" comme action de démarrage, puis lancez l'exécution de l'application.
3. Vérifiez que toutes les actions fonctionnent correctement (Create, Edit, Delete, etc.).

V. Mise à jour du modèle

1. Ajoutez une propriété **Tel** (numéro de téléphone) à la classe **Hotel**.

En lançant l'application, une exception sera déclenchée et affichée sur le navigateur. Elle mentionne qu'il y a un problème avec la nouvelle propriété vu qu'elle n'a pas de colonne correspondante dans la table.



2. Pour rectifier ce problème, il suffit juste de :

- Créer une migration pour la prise en considération de cette nouvelle propriété en lançant la commande *Add-Migration*. Par exemple « **Add-Migration ajoutTel** ». Cette commande créera une classe portant le nom de la migration et contenant, comme précédemment, une méthode **Up()** qui ajoute la colonne **Tel** à la table **Hotels** et une méthode **Down()** qui la supprime.
- Lancer la commande **Update-Database** pour répercuter cette modification sur la BD. Cette commande exécutera la méthode **Up()** de la migration.

Maintenant, la table **Hotels** est à jour, elle contient une colonne dite « **Tel** ».

NB:

- **Si tout va bien, il faut apporter des modifications minimales sur :**
 - **les vues afin qu'elles puissent prendre en considération cette nouvelle propriété;**
 - **les entêtes des méthodes POST des actions Create et Edit.**
 - **Up() et Down() ont toujours des traitements inverses.**
3. Ajoutez dans le dossier **Models\HotellerieModel**, la classe "**Appreciation.cs**" ayant les attributs suivants :
- **Id** (int)
 - **NomPers** (string) : champ obligatoire et doit être affiché sur les vues « Nom Personne ».
 - **Commentaire** (string) : champ obligatoire sous forme d'un texte multi-ligne.
 - **HotelId** (int) : représente l'attribut de la clé étrangère.
4. Ajoutez dans les deux classes de modèle les propriétés de navigation adéquates.
NB : Déclarez-les nullables..
5. Ajoutez dans la classe de contexte la ligne nécessaire pour le *DbSet* correspondant à l'entité "**Appreciation**".
6. Revenez à la console et exécutez les commandes nécessaires pour créer la nouvelle migration et pour mettre à jour la BD.
7. Ajoutez un contrôleur "**AppreciationsController**" sur la classe « **Appreciation** » et testez les différentes actions.
8. Ajoutez un lien hypertexte dans la vue Index des hôtels pour passer à la vue Index des appréciations.
9. Ajoutez un attribut **Note** à la classe "**Appreciation**" :
- **Note** (int)
 - champ obligatoire dont le message d'erreur est à personnaliser;
 - valeur comprise entre 1 et 10.

En créant la nouvelle migration (par exemple **AjoutNote**), la méthode Up() dira que la valeur par défaut de la note est ZERO (voir figure ci-dessous). Cela dit que toutes les appréciations existantes auront une note 0. Or, cette valeur est en dehors de l'intervalle [1, 10].
Donnez une autre valeur par défaut, par exemple 5.

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.AddColumn<int>(
        name: "Note",
        table: "Appreciations",
        nullable: false,
        defaultValue: 0);
}
```

Appliquez les nouveaux changements grâce à la commande "Update-Database".

NB : Il est à rappeler qu'il faut apporter des modifications minimales sur les vues et sur les méthodes POST des actions Create et Edit afin qu'elles puissent considérer ce nouveau champ « Note ».

10. Ajoutez la propriété « **Pays** » à la classe **Hotel**. Puis à partir de la console, créez une nouvelle migration « **AjoutPaysHotel** ». Attribuez la valeur par défaut "Tunisie" à la colonne nouvellement créée « Pays ». Enfin, appliquez cette migration.
11. On voudrait bien revenir à l'état précédent (table "Hotels" sans colonne "Pays"), pour cela, appliquez la commande « **Update-Database -migration AjoutNote** ».

NB : Ici, vous ciblez une ancienne migration. Cela dit, qu'il y aura annulation (exécution des méthodes « Down() ») de toutes les migrations ayant lieu après celle qui est ciblée.

12. On veut bien revenir à l'état initial de la base (avec seulement la table Hotels comme elle a été créée au début). Appliquez la bonne commande.
13. Finalement, on veut revenir au dernier état de la BD avec les deux tables et toutes leurs colonnes. Appliquez la bonne commande.

NB :

- **Exécutez la commande « Get-Migration » pour lister toutes les migrations existantes avec l'état de chacune (appliquée ou non).**
- **Exécutez la commande « Update-Database -Migration 0 » pour revenir à l'état initial (avant toute migration). Cela va exécuter automatiquement la méthode down() de la première migration.**